Software Engineering with Generative AI

Michigan Technological University — Fall 2026 CS 5090 (3 credits, graduate)

Instructor	Dr. Jie J.W. Wu
Email	jie.jw.wu@mtu.edu
Office hours	Mon/Wed 9:30–10:30am, Rekhi 207 — please make an appointment before visiting
Meeting time / room	Mon/Wed/Fri 2:05 – 2:55pm, Fisher 133
Semester	Aug 31 – Dec 11, 2026 (no final exam)

MTU Fall 2026 Key Dates

Per the official MTU 2026–27 academic calendar:

Date	Event
Mon, Aug 31	First day of classes (Week 1)
Fri, Sep 18	Project 2 topics circulated — the research thread starts in Week 3
Fri, Oct 2	**Project 2 research radar due** — solo-or-pair declared, two candidate directions, three papers skimmed
Wed, Oct 7	**Midterm 1**
Fri, Oct 23	Pooled-findings session in class (bring your current Project 1 findings table)
Mon, Oct 26	Project 2 topic pitches (in class)
Wed, Oct 28	Class held **online**
Fri, Oct 30	**No class — deadline day. Project 1 due in full** (one deadline: all four phases + demo video) · **Project 2 proposals due** — topic pivots allowed until Fri, Nov 20
Wed, Nov 18	**Midterm 2** (covers Module 2) — the latest slot MTU exam policy allows a clear run after
Fri, Dec 4	Project 2 full paper drafts due
Mon & Wed, Dec 7–9	**Project 2 design presentations + preliminary results** (in class)
Fri, Dec 11	Last day of classes; **Project 2 final paper due**
Dec 14–18	Finals week — **no final exam in this course**

Holiday class cancellations are noted in the Course Schedule below. Exact midterm days will be adjusted to the actual meeting pattern once scheduled; add/drop and withdrawal deadlines follow the Registrar's calendar.

Course Description

Generative AI — and large language models (LLMs) in particular — is materially changing how software is built. This course examines that change from both directions: **software engineering *of* generative AI systems** (building, evaluating, and operating systems with learned components) and **software engineering *with* generative AI** (AI-assisted and agentic development workflows).

AI-assisted and agentic software engineering is not a standalone unit at the end — it is a **thread that runs through the whole course**. From Week 1, students use AI coding assistants and agents as their required development workflow (and learn to validate their output); the foundations module explains how those tools work by building one; the production module covers how to engineer, evaluate, and operate LLM-powered and agentic systems. The second half of the course runs as a research seminar: students — solo or in pairs — carry out an original research project and write it up as a conference-format paper, which is submitted to a suitable venue after the semester ends.

Prerequisites

- Probability and statistics, linear algebra, data structures
- Programming maturity (Python); prior exposure to machine learning is helpful but not required
- Graduate standing, **or instructor permission for undergraduates**

Undergraduate students

This is a graduate course, and a small number of undergraduates take it with instructor permission. They do the same lectures, the same Project 1, and the same exams. Three adjustments apply, so that the graduate-level research expectations do not fall on a student who has not been taught them yet:

	Adjustment
Project 2 pairing	Project 2 is done solo or in pairs; each undergraduate **pairs with a graduate student**. No solo undergraduates, and no undergraduate-only pairs.
Design presentation	Undergraduates **co-present** the design presentation with their graduate partner rather than presenting alone.
Reading load	One required paper per week rather than two; the second is optional.

Everything else — Project 1 in full, both midterms, the findings table, the Week 8 pooled-findings session — is identical. Project 1 is deliberately scaffolded phase by phase and does not assume research experience, so it is the right entry point regardless of level.

The IJCAI bonus is open to undergraduates on the same terms as everyone else.

Learning Outcomes

Upon successful completion of this course, students will be able to:

1. Explain at a principled level how large language models are trained (pretraining, instruction tuning, RLHF/preference optimization) and how they generate output.
2. Build, fine-tune, and evaluate a small code-generation language model end to end, including data preparation, tokenization, training, and benchmark evaluation.
3. Analyze training datasets for provenance, bias, contamination, licensing, and governance implications.
4. Explain embeddings as a representational framework and apply them to retrieval and similarity tasks (e.g., retrieval-augmented generation over codebases).
5. Evaluate AI as a system component with defined cost, latency, reliability, safety, and privacy characteristics, and design systems that mitigate common failure modes (hallucination, adversarial inputs, miscalibration, prompt injection).
6. Apply production ML practices — requirements, architecture, deployment, testing, monitoring, and MLOps — to systems with LLM components.
7. Effectively use AI coding assistants and AI agents within a disciplined, collaborative development workflow (version control, code review, validation of AI-generated code), and design and evaluate agentic systems for SE tasks.
8. Critically read and discuss current research in AI-assisted and agentic software engineering.
9. Design and execute an empirical software engineering study involving generative AI, and communicate the results in a conference-quality research paper.

Course Materials

Two textbooks, used deeply. T1's companion code (a complete, runnable build of a GPT-style model) is free on GitHub and the book is available through the MTU library; T2 is freely readable online. Every other reading is a research paper available via arXiv or the ACM/IEEE digital libraries (through the MTU library).

Textbooks

#	Text	Used for
T1	S. Raschka, *Build a Large Language Model (From Scratch)*, Manning, 2024 — code: https://github.com/rasbt/LLMs-from-scratch	Module 1 and Project 1: tokenization, attention, the GPT architecture, pretraining, fine-tuning — the same build-it path the course takes
T2	C. Kästner, *Machine Learning in Production: From Models to Products*, MIT Press — https://mlip-cmu.github.io/book/	Module 2: requirements, architecture, quality assurance, operations, responsible AI

The ⚙ AI-assisted / agentic SE thread is grounded directly in the research literature (e.g., Hassan et al.'s agentic-SE roadmap below) rather than a textbook — the field moves faster than books do.

Research papers

Each week's readings pair a textbook chapter with 1–2 papers. The recurring set below is a summary; the authoritative, per-slide reference list is the citation registry in `slides_module_1_v3/template/refs.py`

(100 papers and books), and every slide that makes a specific claim shows its source at the foot of the slide.

- **Code LLMs:** Chen et al., *Evaluating Large Language Models Trained on Code* (Codex/HumanEval, 2021) · Li et al., *Competition-Level Code Generation with AlphaCode* (2022) · Li et al., *StarCoder* (2023)
- **Agents:** Yang et al., *SWE-agent* (2024) · Jimenez et al., *SWE-bench* (2024) · Wang et al., *OpenHands* (2024)
- **Reasoning & tools:** Wei et al., *Chain-of-Thought Prompting* (2022) · Gao et al., *PAL: Program-Aided Language Models* (2022) · Schick et al., *Toolformer* (2023)
- **Post-training:** Ouyang et al., *InstructGPT* (2022) · Rafailov et al., *Direct Preference Optimization* (2023) · Hu et al., *LoRA* (2021)
- **Human factors & security:** Peng et al., *The Impact of AI on Developer Productivity* (2023) · Perry et al., *Do Users Write More Insecure Code with AI Assistants?* (2022) · Minelli et al., *I Know What You Did Last Summer* (developer time use, 2015)
- **Vision & roadmap:** Hassan et al., *Agentic Software Engineering: Foundational Pillars and a Research Roadmap* (arXiv:2509.06216)

Grading

Component	Weight
Midterm exam 1 (Module 1)	10%
Midterm exam 2 (Module 2)	10%
Participation & in-class engagement	10%
Project 1 — Build it, use it, break it, assure it	25%
Project 2 — Research project (conference submission)	45%
Bonus — IJCAI 2027 submission (instructor-approved)	**+5%**

There is no final exam. The two midterms are short, in-class exams: Midterm 1 covers the Module 1 foundations decks (`v3-L00`–`v3-L06`), Midterm 2 covers the Module 2 decks. The ⚙ AI-assisted/agentic SE sessions are practice-oriented and are assessed through the projects and participation, never on an exam. Each examinable deck ends with a 'Key takeaways' slide; those slides are the examinable core.

Bonus credit (+5% of the course grade). A team whose final paper is **reviewed and approved by the instructor** submits it to **IJCAI 2027** (deadline ~Feb 1, 2027; abstracts ~Jan 8) and earns bonus credit for every team member. The bonus is awarded for a completed, instructor-approved submission — **not** for acceptance, which is decided long after grades are due. Approval means the paper meets a real venue's bar: a falsifiable research question, a fair baseline, reported variance, an honest threats-to-validity section, and compliance with IJCAI's formatting and AI-disclosure rules. Teams that are not approved lose nothing; the bonus only ever adds.

Participation & in-class engagement (10%)

- Reading notes: for weeks with assigned papers, submit a short note (2–3 questions or critiques) before class.
- Presentation discussant: every student asks at least one substantive question during another team's Week 14 design presentation.
- Peer review: each student writes mock program-committee reviews of two other teams' paper drafts (drafts circulate Fri, Dec 4; reviews due Mon, Dec 7).
- In-class hands-on sessions, the Week 8 pooled-findings session, and general engagement.

Project 1 — Build It, Use It, Break It, Assure It (25%, individual, Weeks 1–9)

Full spec: `project1/`

The question Project 1 answers: *how should software engineers be trained when AI is part of the process?* The course's answer is that you make them build the component, use it for real work, attack it, and then engineer the checks that catch what they found.

One deadline: everything is due Fri, Oct 30 — one bundle: model card + repo, task report, findings table, QA report + a **3-minute recorded demo video**, AI-use log + reflection. The four phases have a **recommended pace**, not graded deadlines:

Phase	Recommended pace	What you do	Hand in (Oct 30)
1 · Build	by Fri Sep 25	Train a small code model with nanoGPT	Model card + repo
2 · Use	by Fri Oct 9	Use *your own* model on a real SE task, on 20 real inputs	Task report
3 · Break	by Fri Oct 16	Red-team it — find 10+ distinct failures	**Findings table**
4 · Assure	by Fri Oct 23	Build checks that catch them; measure your catch rate	QA report + demo video

The pace exists because the later phases cannot be compressed into the last weekend — and because the **Week 8 pooled-findings session (Fri, Oct 23)** works from whatever findings table you bring that day. Everyone uses the same eight failure categories, so the class ends the term with a pooled catalogue of how a code model fails and how much of that ordinary testing catches; the final catalogue is assembled from the Oct 30 submissions.

The AI-assisted workflow is **required**, with a log and a one-page reflection.

Graded on what you found and what you built to catch it — not on model quality. A model that produces broken Python is fine; it gives you more to find in Phase 3. Rubric: `project1/rubric.md`.

A note on data. Anonymised Phase 3 findings may be used in a study of how this course is taught. Participation is voluntary, opt-in via a Week 1 consent form, and has no bearing on grades in either direction.

Compute: ACCESS Classroom allocation (Explore), NAIRR Pilot as a complement — `project1/compute.md`.

Project 2 — Research Project with Conference Submission (45%, solo or in pairs, Weeks 3–14)

Full spec: `project2/` · templates for the proposal and the mock-PC review are in `project2/templates/`; candidate venues in `project2/venues.md`.

An original research project on software engineering with/for generative AI, carried out **solo or in pairs** and written up as a conference-format paper. The goal is to take one question from idea to evidence to a paper an outside reviewer would take seriously — and then actually submit it. The rubric is the same for one student as for two; scope expectations scale with the number of hands, and the proposal is where scope is agreed. Undergraduates pair with a graduate student (see above).

The research thread starts in Week 3. Topics circulate on **Fri, Sep 18**, and by **Fri, Oct 2** each student files a half-page **research radar**: solo-or-pair declared, two candidate directions, and three papers actually opened, one sentence each. The point is to put you in research mode early — reading the literature in September, while Project 1 is teaching you the craft. Your Phase 3 findings table will later sharpen the choice (revisit your radar after red-teaming — several of the best questions come from there). Full-time execution begins Nov 2, giving **five working weeks** before the draft. Many projects extend the Project 1 experience directly — four of the circulated candidate topics reuse the class's own models, probes, and QA checks.

Why we submit to a real venue. Writing for an external audience is the pedagogy, not a trophy hunt. A real deadline and real reviewers force the things this course is about: a question you can actually answer, an honest baseline, results you can defend, and limitations you state yourself. Teams that make it that far learn more than any term paper could teach.

How submission works. Papers are **submission-ready on Dec 11**; teams may post an arXiv preprint then. Over the winter break the instructor works with each team to pick a venue whose scope and deadline fit their work — options are discussed in Week 12 and tracked in `project2/`, and the deadlines that suit a December finish fall between January and March. **Papers the instructor reviews and approves are submitted to IJCAI 2027 for +5% bonus credit** (see Grading).

Grading is independent of the outcome. Your grade reflects the quality of the research and the writing — the study you designed, the evidence you gathered, the honesty of your analysis. Acceptance decisions arrive months after grades are final and play no part in them. A rigorous negative result earns full marks.

Milestones (weights are of the Project 2 grade):

Date	Milestone	Weight
Fri, Sep 18	Topics circulated (`project2/topics.md`) — the research thread opens	—
Fri, Oct 2	**Research radar** — half a page: solo-or-pair, two candidate directions, three papers skimmed	5
Mon, Oct 26	Five-minute topic pitch, in class	5
Fri, Oct 30	Written proposal (problem, method, baseline, evaluation plan) — topic pivots allowed until Fri, Nov 20	15

Date	Milestone	Weight
Nov 2 – Dec 4	Execution — five working weeks; the ⚙ advanced sessions run alongside; check-ins in office hours	—
Fri, Dec 4	Full paper draft	10
Mon, Dec 7	Two written mock-PC peer reviews	10
Dec 7 / Dec 9	Design presentation + preliminary results	15
Fri, Dec 11	Final paper + artifact, submission-ready	40
after	Instructor review over break → venue chosen with each team; approved papers submitted to **IJCAI 2027** for bonus credit	+5% course bonus

---

Course Schedule

Two teaching modules with the ⚙ AI-assisted/agentic SE thread woven through both, then a flexible design-review block. Three 50-minute lectures per week (Mon/Wed/Fri) when there is no holiday. The **Decks** column names the slide files; ×2 means two sessions; ⚙ marks a practice session taught from the `slides_module_3/` thread decks — supporting the projects, never examined.

> **Holiday cancellations (MTU academic calendar):** no class **Mon, Sep 7** (Labor Day) — Week 2 has **two sessions** — and no classes the week of **Nov 23–27** (Thanksgiving). Marked ⛔ below.

Module 1 — Foundations: Build a Code LM, with the ⚙ thread (Weeks 1–8)

Slides: `slides_module_1_v3/` (10 decks) plus three ⚙ sessions from `slides_module_3/`. Each ⚙ session lands just before the Project 1 phase it supports.

Week	Dates	Sessions	Decks	Projects
1	Aug 31 – Sep 4	Course overview · ⚙ **hands-on: assistant, Git, compute account, AI-use log** · what a language model is	`v3-L00` · `v3-L01` (1 of 2)	**Phase 1 released**
2	Sep 8 – 11	Attention and the Transformer block · position, norms, model families	`v3-L01` (2 of 2) · `v3-L02`	⛔ Labor Day — two sessions
3	Sep 14 – 18	Decoding, guided output, KV cache, MoE, prompting ×2 · ⚙ **building with AI assistance** — context engineering, decomposition, plan-then-act, for the required workflow	`v3-L03` ×2 · ⚙ `m3-L03`	training runs start · **P2 topics circulated Fri Sep 18**

Week	Dates	Sessions	Decks	Projects
4	Sep 21 – 25	Pretraining, data mix, scaling laws, SFT and LoRA ×2 · Project 1 clinic	`v3-L04` ×2	Build pace: Sep 25 (recommended)
5	Sep 28 – Oct 2	Preferences, reward models, DPO · reasoning, test-time scaling, GRPO · ⚙ **the new workflow** — from author to reviewer, levels of autonomy, what the evidence shows	`v3-L05` · `v3-L06` · ⚙ `m3-L01`	**P2 radar due Fri Oct 2** — solo/pair + directions + 3 papers
6	Oct 5 – 9	Midterm review · **Midterm 1 (Wed, Oct 7)** · Phase 2 debrief	—	Use pace: Oct 9 (recommended)
7	Oct 12 – 16	Retrieval over a repo, tool calling, MCP, the agent loop ×2 · ⚙ **agentic SE** — a worked episode, tool design, SWE-bench, failure modes (highlights)	`v3-L07` ×2 · ⚙ `m3-L06`	Break pace: Oct 16 (recommended)
8	Oct 19 – 23	Evaluation: pass@k, benchmarks, contamination · ⚙ **testing with LLMs** — the oracle problem, TDD, mutation score, for the Assure phase · **pooled-findings session** (bring your current table) + module wrap	`v3-L08` · ⚙ `m3-L04` · `v3-L09`	Assure pace: Oct 23 (recommended)

Midterm 1 covers `v3-L00` – `v3-L06`. The ⚙ sessions are not examined.

Module 1 — weekly key content and readings

T1 (Raschka) is read in course order and **finished by Week 5, on purpose** — from Week 7 the readings are research papers, because that is where the field lives. Bold marks the week's one idea to not miss.

Week	Key content	Read
1	What a language model is — tokens, embeddings, next-token prediction	T1 ch. 1–2
2	**Attention and the Transformer block** — the core idea of the course	T1 ch. 3–4 · Vaswani et al. 2017
3	Decoding and prompting — the model as an interface you tune at run time	T1 ch. 5 · Chen et al. 2021 (Codex)
4	**Pretraining and scaling** — where capability comes from; LoRA	T1 ch. 5–6 + App. E · Hoffmann et al. 2022 · Hu et al. 2021
5	Instruction following and preferences; reasoning models	T1 ch. 7 · Ouyang et al. 2022 · Rafailov et al. 2023 · DeepSeek-R1
6	**Midterm 1** — revise from the Key-takeaways slides of `v3-L00 – L06`	—
7	**Beyond the textbook** — retrieval, tool calling, the agent loop	Lewis et al. 2020 · Yao et al. 2023 (ReAct) · Jimenez et al. 2024 (SWE-bench)

Week	Key content	Read
8	Evaluating code models honestly — pass@k, benchmarks, contamination	Chen et al. 2021 §2.1 · one leaderboard's contamination statement

Module 2 — Machine Learning in Production, with the ⚙ thread (Weeks 9–11)

Slides: `slides_module_2/` (8 decks) with three ⚙ pairings from `slides_module_3/` — each ⚙ deck taught alongside the production topic it extends, selecting from both.

Week	Dates	Sessions	Decks	Projects
9	Oct 26 – 30	**P2 pitches (Mon)** · the model is a small box (Wed, **held online**) · ⛔ **Fri: no class — deadline day**	`m2-L01`	**P1 due in full + P2 proposal due Fri Oct 30**
10	Nov 2 – 6	When ML is the right tool, goals and measurement · requirements and mistakes **+ ⚙ requirements in the AI era** (paired, select) · architecture, deployment, MLOps	`m2-L02` · `m2-L03` + ⚙ `m3-L02` · `m2-L04`	**experiments start Mon Nov 2** — proposal feedback returned
11	Nov 9 – 13	Four-level QA **+ ⚙ assuring generated code** (paired, select) · testing in production, versioning · process, teams, debt **+ ⚙ productivity** and responsible engineering (selections)	`m2-L05` + ⚙ `m3-L05` · `m2-L06` · `m2-L07` + ⚙ `m3-L07` · `m2-L08`	experiments running · team check-ins in office hours

Midterm 2 covers the `m2` decks. The ⚙ pairings are not examined.

Module 2 — weekly key content and readings

T2 (Kästner) is read by chapter cluster, matching the decks; each ⚙ pairing adds one paper from the slide citations.

Week	Key content	Read (T2 chapters)
9	**The model is a component** — systems thinking around a learned part	From Models to Systems
10	Goals, requirements, **planning for the model being wrong**; architecture	when-to-use-ML and goal-setting chapters · requirements + planning-for-mistakes chapters · architecture/deployment/MLOps chapters
11	**Quality assurance at four levels**; production; teams and responsibility	model-quality + data-quality chapters · testing-in-production chapter · process/teams/debt + responsible-engineering chapters

Weeks 12–14 — Midterm 2, advanced topics, then the Project 2 finish

The end of the course runs: **exam → advanced SE topics → draft → design presentation with preliminary results → final paper.** The advanced-topic sessions are flexible — they are taught from the ⚙ deep-dive material and current papers, and any of them yields to a Project 2 clinic when teams need it.

Session	Plan
Mon, Nov 16	Midterm 2 review session
Wed, Nov 18	**Midterm 2** — covers the Module 2 decks, taught through Fri Nov 13
Fri, Nov 20	⚙ **Advanced topic 1** — agentic SE deep dive (`m3-L06` in full)
⛔ Nov 23 – 27	*Thanksgiving — no classes*
Mon, Nov 30	⚙ **Advanced topic 2** — AI for testing and assurance, in depth (`m3-L04` / `m3-L05` material)
Wed, Dec 2	⚙ **Advanced topic 3** — instructor's pick from the current literature, or by class vote
Fri, Dec 4	Paper clinic · **draft due (end of day)**
Mon, Dec 7	**Design presentations + preliminary results**, first block (8 min + 2 questions; five talks per session)
Wed, Dec 9	**Design presentations + preliminary results**, second block
Fri, Dec 11	Course wrap — the professional stance (`m3-L07`) · **final paper + artifact due**

Why the advanced topics sit there. They are cutting-edge material — agents, AI-for-testing, whatever the literature is doing this month — taught when exams are over and students are deep in their own research. They sharpen Project 2 work in flight, and they seed next steps (and next-cohort topics). They are never examined.

Why the presentation comes after the draft. Presenting design *plus preliminary results* means every talk has substance, and the feedback — live questions plus the written peer reviews — arrives exactly one revision week before the final deadline, when teams can still act on it.

Not comfortable presenting your own idea? That is allowed for. If you would rather not share your novel idea publicly before submission, you may instead present **one closely related paper, end to end** — its problem, method, evidence, and limitations, plus one slide on how it connects to your project's area. Same slot, same length, same rubric; tell the instructor by the Fri Dec 4 draft deadline so the sessions can be ordered.

Written peer reviews: drafts circulate Friday evening (Dec 4); each student reviews two other teams' drafts by **Mon Dec 7**, using `project2/templates/peer-review.md`. Discussed in office hours.

Where the slack is — and is not

The semester has **40 class meetings** (Fri Oct 30 is a deadline day, not a class): 23 in Module 1's weeks, 8 in Module 2's — one of them online — and 9 in the Weeks 12–14 finish block. The advanced-topic sessions (Nov 20 – Dec 2) are the slack: each one yields to a Project 2 clinic, an overflow presentation slot, or a snow-day catch-up before anything else moves. Everything before Week 12 is committed.

Presentations run five talks per 50-minute session (8 min + 2 questions), so Dec 7 and Dec 9 hold ten. With solo-or-pair teams a class of twenty can produce up to fifteen talks: talks 11–15 go to **Wed Dec 2**, whose advanced-topic slot converts to a third presentation session, and only past fifteen does the Fri Dec 4 clinic yield. The final Friday stays free for the wrap and the deadline.

A voluntary demo hour in finals week (Dec 14–18) remains available — this course has no final exam.

Each deck folder holds more material than its sessions allow; the decks are built to be selected from, not marched through.

Course Policies

Use of AI tools

Using AI assistants (Copilot, Claude, ChatGPT, agents, etc.) is **required for Project 1 and encouraged everywhere else** — disciplined use of these tools is a learning outcome of this course. Requirements:

- **Disclose** AI use in every deliverable (which tools, for what, and how outputs were validated); Project 1 additionally requires an AI-use log.
- **You are accountable** for everything you submit: AI-generated code and text must be reviewed, tested, and understood. "The AI wrote it" is not a defense for plagiarized, incorrect, or insecure work.
- AI tools are **not permitted during the two midterm exams**.
- For Project 2, follow the target venue's policy on AI-generated text in submissions.

Late work

Project milestones lose 10% per day late (max 3 days) unless arranged in advance. The Week 13 draft and Week 14 final deadlines are firm because peers and the submission timeline depend on them.

Academic integrity

All work is governed by the MTU Academic Integrity Policy. Collaboration within teams is expected; representing others' (or undisclosed AI) work as your own analysis is not.

Accessibility and wellbeing

MTU complies with the ADA and Section 504. Students with disabilities should contact Student Accessibility Services and the instructor early in the semester. Standard MTU syllabus statements (accessibility, non-discrimination, Title IX, mental health resources) apply and will be linked on Canvas.
